

Stealthy and Robust Glitch Injection Attack on Deep Learning Accelerator for Target With Variational Viewpoint

Wenye Liu^{ID}, *Graduate Student Member, IEEE*, Chip-Hong Chang^{ID}, *Fellow, IEEE*,
and Fan Zhang^{ID}, *Member, IEEE*

Abstract—Deep neural network (DNN) accelerators overcome the power and memory walls for executing neural-net models locally on edge-computing devices to support sophisticated AI applications. The advocacy of “model once, run optimized anywhere” paradigm introduces potential new security threat to edge intelligence that is methodologically different from the well-known adversarial examples. Existing adversarial examples modify the input samples presented to an AI application either digitally or physically to cause a misclassification. Nevertheless, these input-based perturbations are not robust or surreptitious on multi-view target. To generate a good adversarial example for misclassifying a real-world target of variational viewing angle, lighting and distance, a decent number of target’s samples are required to extract the rare anomalies that can cross the decision boundary. The feasible perturbations are substantial and visually perceptible. In this paper, we propose a new glitch injection attack on DNN accelerator that is capable of misclassifying a target under variational viewpoints. The glitches injected into the computation clock signal induce transitory but disruptive errors in the intermediate results of the multiply-and-accumulate (MAC) operations. The attack pattern for each target of interest consists of sparse instantaneous glitches, which can be derived from just one sample of the target. Two modes of attack patterns are derived, and their effectiveness are demonstrated on four representative ImageNet models implemented on the Deep-learning Processing Unit (DPU) of FPGA edge and its DNN development toolchain. The attack success rates are evaluated on 118 objects in 61 diverse sensing conditions, including 25 viewing angles (-60° to 60°), 24 illumination directions and 12 color temperatures. In the covert mode, the success rates of our attack exceed existing stealthy adversarial examples by more than 16.3%, with only two glitches injected into ten thousands to a million cycles for one complete inference. In the robust mode,

the attack success rates on all four DNNs are more than 96.2% with an average glitch intensity of 1.4% and a maximum glitch intensity of 10.2%.

Index Terms—Deep learning accelerator, fault injection attack, adversarial misclassification, image recognition.

I. INTRODUCTION

EDGE artificial intelligence (AI) is an enabler of a variety of dazzling deep learning applications like personal assistant, smart home appliance and video surveillance [1]. Two essential components that drive AI applications on edge devices are the energy-efficient deep neural network (DNN) accelerator [2] and the open-source AI-model compilers [3]. DNN accelerators are domain-specific computing platforms focusing on specific customization for deep learning for various edge-target platforms. The incubating open source deep learning compiler stack and related toolchain aim to optimize AI models automatically for fast and energy-efficient edge execution, and adapt them to evolving hardware and software without compromising the accuracy for AI application development. The over-hyping of edge AI and the clamoring about the omnipotence of DNN, however, can be dangerous. With the growing adoption of edge AI in privacy (e.g., face recognition system for self-service check out) and safety-critical tasks (e.g., self-driving car), edge AI becomes an attractive target for malicious attacks. An important associated threat of AI attacks is that the scope of cyberattacks can be expanded with the human supervision element taken over by the AI.

What make AI attacks unique is that the exploitable vulnerability is neither program “bugs” nor human mistakes, but the way the underlying deep learning models learnt to become extremely proficient at one singular task. The fact that both robust and non-robust features exist inherently in the training dataset used by supervised learning makes it possible to add offensive noise into an input, known as adversarial examples, to force a well-trained DNN classifier to produce a predication that is obviously wrong and unanticipated by a human being [4]. Fast adversarial example generation algorithm [5] and human imperceptible adversarial examples [6]–[8] have been reported. They can mislead the AI with high probability of success because the classifier cares only about the accuracy determined by the label of the input during supervised learning. All features, comprehensible and incomprehensible, perceptible and non-perceptible to a human observer, are

Manuscript received July 10, 2020; revised November 10, 2020; accepted December 9, 2020. Date of publication December 23, 2020; date of current version January 8, 2021. This work was mainly supported by the National Research Foundation, Singapore, through the National Cybersecurity R&D Programme/Cyber-Hardware Forensic & Assurance Evaluation R&D Programme under Award CHFA-GC1-AW01, and in part by the National Key R&D Program of China (2020AAA0107700), National Natural Science Foundation of China (62072398), Zhejiang Key R&D Plan (2019C03133), and Alibaba-Zhejiang University Joint Institute of Frontier Technologies. The associate editor coordinating the review of this manuscript and approving it for publication was Prof. Georg Sigl. (*Corresponding author: Chip-Hong Chang.*)

Wenye Liu and Chip-Hong Chang are with the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore 639798 (e-mail: wliu015@e.ntu.edu.sg; echchang@ntu.edu.sg).

Fan Zhang is with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310027, China, and also with the Alibaba-Zhejiang University Joint Research Institution of Frontier Technologies, Hangzhou 310027, China (e-mail: fanzhang@zju.edu.cn).

Digital Object Identifier 10.1109/TIFS.2020.3046858

1556-6021 © 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

meaningful to the machine. However, which features of an input the algorithms learnt to maximize its accuracy are left entirely to brute force tuning of the weight parameters of the model.

Vision based object recognition has been widely adopted in many safety-sensitive applications such as access control and autonomous vehicles, as well as privacy-sensitive technologies such as surveillance robots, point-of-sale kiosks and automatic teller machines. Live images captured in a real scene for these applications are easily affected by multiple factors involving the subject or the environment. For instance, when a person interacts with the camera of a face recognition system in an outdoor environment, the viewing angle of its image is based on the relative orientation between the person and the camera. The misclassification rate of adversarial examples crafted in a specific ambience can be more severely impacted by adverse lighting conditions such as glares and shadows than the correct detection rate of the original inputs. Sunlight can vary in color, direction, inclination and intensity in different weather and in different time of the day. Artificial indoor lightings and streetlight illumination at night can also increase the non-uniformity of an input distribution. Consequently, notwithstanding the good transferability of adversarial examples over different models, a perturbation-minimized adversarial example of a target has limited transferability over dynamic context variations such as changes in viewing angle, scene illumination and distance of the target [9].

Recent studies show that special shaped adversarial examples, such as eyeglass frames [10], synthetic stickers [11]–[13] and 3D-printed samples [14], are able to fool deep learning algorithms from multiple viewpoints. As the statistical performance of DNN is largely determined by the coverage of training dataset, these extended physical adversarial attacks draw samples from realistic distribution to find the valid perturbations for each target under different scenarios [11], [14], [15]. Collecting adequate samples of a target under diverse external factors in a real scene is extremely hard and impractical, as the attacker has no control over the training of the subjects like the designer of the DNN. The attacker is often limited by the number of live samples of the target that can be secretly acquired to generalize the attack vector over various non-uniformities of imaging conditions. With only a decent set of samples from different sensing conditions, the areas of perturbations ($\sim 25\%$ [13]) on the original input are large and conspicuous, yet most may not be capable of crossing the original target decision boundary after propagating through dozens to hundreds of layers.

AI-specific accelerator for DNN model implementation in edge scenario offers a different opportunity for an attacker to directly intervene the processing of deep learning model without changing the input or tampering the target physically. Hardware-oriented attacks can perturb the DNN by skipping instructions [16], flipping weights [17], [18] and infecting operations [19], [20]. Albeit effective, many of these techniques such as laser beam [16], memory collision [21] and rowhammering [22] are either invasive, not stealthy or can result in partial or permanent performance degradation. Recently, a stealthy fault-injection attack on deep learning accelerator has been proposed [23]. It injects

infrequent glitches into the clock signal to corrupt intermediate results of convolution computations. The fault occurs instantaneously on the internal processing and leaves no trace for detection after the incorrect inference. This attack, like other existing hardware-based attacks, did not address the dynamic scene variations of the target.

In this paper, we propose a new robust attack on deep learning accelerators which is capable of stealthily misclassifying a target under a variety of viewing conditions. The proposed attack is founded on our preliminary glitch injection attack [23], with enhanced stealth and versatility. Most importantly, a generalized and systematic formal approach has been developed to profile the timing trigger of a target for a least glitch attack (LGA) or a sensitive layer attack (SLA) on a DNN. Comparing with [23], this paper has been strengthened with the following distinctive contributions:

1. A comprehensive evaluation on object misclassification rate with changing environments of advanced digital adversarial attacks, including FGSM [5], Iterative FGSM [6], CW [7] and EADL1 [8], has been performed. We show that adversarial examples with perceptibility-minimized perturbations have low generalization ability (with the same perturbations for multiple viewpoints), and the relative attack performance shares the similar trend as their transferability (with the same perturbations for different models) [24].
2. We extend our preliminary stealthy misclassification attack concept [23] by increasing the attack robustness beyond a fixed viewing condition of the target where the attack pattern is derived. Two new layer-wise glitch injection methods are proposed to fool a well-trained DNN classifier under diverse imaging scenarios of target's viewing angles, surrounding illuminations and distances.
3. We demonstrate our attack on FPGA-based deep learning accelerator implementation of four representative ImageNet models, including ResNet50 [25], Inceptionv4 [26], MobileNetv2 [27] and VGG16 [28]. 61 viewing conditions are tested, including 25 orientations, 24 lighting directions and 12 illumination temperatures from Amsterdam Library of Object Images (ALOI) [29]. This is significantly more comprehensive and convincing than the variational viewpoint attacks reported in existing physical adversarial examples [11], [13]–[15] for a limited scenario from self-collected images. Our most robust SLA has achieved a success rate of at least 96.2% for all evaluated viewing conditions. With only two glitches injected into ten thousand to a million clock cycles, the success rates of LGA on all targets evaluated also well exceeded the state-of-the-art stealthy adversarial algorithms by at least 16.3%.
4. Our proposed algorithms can automatically extract an attack pattern to maximize either the secrecy or the robustness of a misclassification using only one input sample of the target. Comparing with both digital and physical adversarial examples, our investigation shows that hardware-based fault-injection attacks have extra degree of freedom to falsify intermediate data directly. Therefore, they can achieve better results without having

to acquire enough number of target images to extrapolate the features and decision boundary of the target in different environmental contexts used by the machine to maximize the classification accuracy.

The rest of the paper is organized as follows. Hardware acceleration on deep learning, adversarial example attacks and fault attacks on DNN hardware are introduced in Section II. The attack model and our approach are described in Section III. Section IV presents the dataset, DNN models, design of experiments before the evaluation results are presented and compared. A reflection of input-based and process-interpolated perturbations, and the trade-offs in potential countermeasures against hardware-based attacks are further discussed in Section V. Section VI concludes the paper.

II. PRELIMINARIES AND RELATED WORKS

A. Deep Learning and Hardware Accelerator

Deep learning delivers superior performance on many AI applications, especially for vision recognition problems [30]. This outstanding capability comes with a demand for enormous data and computing power. Among the operations of DNN algorithm, those incurred in the convolutional layer and fully-connected layer are most compute-intensive. A single step of convolution is implemented by applying a filter to the receptive field of an input as follows:

$$y = \sum_{i=1}^K \sum_{j=1}^K \sum_{d=1}^D w_{i,j,d} \times a_{i,j,d} + \mu \quad (1)$$

where K is the filter height/width, D is the filter depth, w is the filter weight, a is an input element and μ is the bias.

Since y is only an element of the output feature map of a convolution layer, the total computation cost of a convolution layer is that of a single step of convolution in (1) multiplied by the height, width and depth of the output feature map.

The output features b of a fully-connected layer can be computed by a matrix-vector multiplication as follows:

$$b = Wz + \mu \quad (2)$$

where z is the input vector and W is the weight matrix. Output features of convolutional layer and fully-connected layer will be activated by a non-linear function (e.g., ReLU) before propagating to the next computational layer.

From (1) and (2), the fundamental component in both the convolutional layer and the fully-connected layer is the multiply-and-accumulate (MAC) unit. Recent DNN models consume from hundred millions to billions of MACs per image for inference. This enormous amount of computing power can only be met by specialized hardware accelerator for energy and resource-constrained embedded devices [2]. The energy efficiency can be achieved through data quantization or approximation [3], while adequate throughput can be attained by parallelizing and pipelining the MAC operations [31], [32]. Quantizing single precision floating-point data into fixed-point (e.g., 8 bits) representation saves not only a large proportion of energy on expansive off-chip memory accesses [31], but also the power consumption on each MAC operation [33].

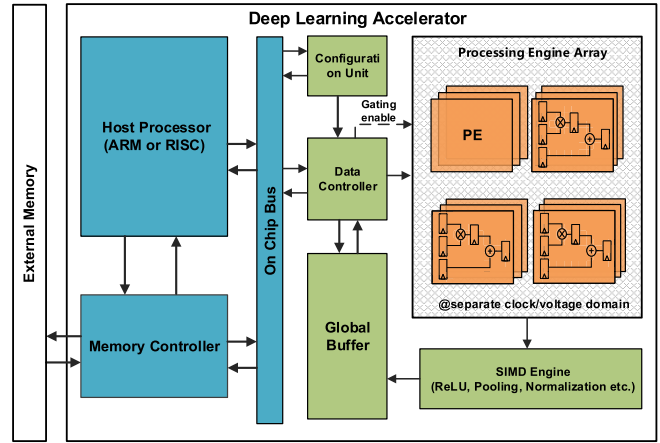


Fig. 1. Typical architecture of a DNN accelerator. The orange blocks of the PE array run on separate clock/voltage domain to facilitate DVFS power management.

To boost the throughput, the MAC operations are executed parallelly in an array of processing engines (PEs) with a high-frequency clock [34]. A typical architecture of deep learning accelerator is illustrated in Fig. 1. A host processor (e.g., ARM or RISC) is embedded as a control unit to schedule the DNN operations. The most compute-intensive tasks such as convolution and matrix-vector multiplication are performed in the PE array. Each PE is a MAC unit composed of a fixed-point multiplier and an accumulator. The remaining operations like ReLU, Pooling and Normalization are performed on the single instruction multiple data (SIMD) engine [33]. The on-chip dataflow is optimized through the data controller by reducing the external memory accesses [35]. The PE array runs on separate clock [34] and supply voltage [33] domains to enable dynamic voltage-frequency scaling (DVFS). The operating clock frequency and the supply voltage are scaled in such a way that the Quality-of-Service (QoS) requirement is met with reasonably reduced power consumption. The clock domain of the PE array can also be turned off by the gating enable signal when the PE array is in the idle mode [34].

B. Digital and Physical Adversarial Examples

DNN classifiers are known to be vulnerable to misclassification by adversarial examples, which are carefully crafted perturbation added to the input [4]. Various algorithms have been proposed to generate adversarial examples to change each data point in the digital input space to an arbitrary value. Fast gradient sign method (FGSM) [5] and its generalized iterative version (I-FGSM) [6] make use of the gradient of the loss function to create the adversarial examples. The latter can be expressed by the following recursion:

$$x'_s = \text{clip}_{x'_0, \epsilon} \{x'_{s-1} + \alpha \cdot \text{sign}(\nabla \text{loss}(x'_{s-1}, c_o))\} \quad (3)$$

where x'_0 is the clean input image, c_o is the correct label, s is the iterative step, ϵ is the parameter to constrain L_∞ norm and α is the step size of each iteration. FGSM is a special case of I-FGSM where $s = 1$ and $\alpha = \epsilon$.

Carlini and Wagner (CW) [7] and L1-oriented Elastic-net Attacks to DNNs (EADL1) [8] generate visually imperceptible

perturbation δ in an original image x by:

$$\begin{aligned} & \text{minimize } c \cdot f(x + \delta, \kappa) + \|\delta\|_p + \beta \|\delta\|_1 \\ & \text{subject to } x + \delta \in [0, 1]^n \end{aligned} \quad (4)$$

where c is a constant larger than 0, $\|\cdot\|_p$ is the L_p Norm (p can be 0, 2 or ∞). $\beta \geq 0$ is the regularization parameter that controls the L_1 penalty, which is adopted only in EADL1 [8]. An appropriate objective function $f(\cdot)$ is selected¹ for solving (4), where κ is an adjustable confidence parameter. A larger κ implies that a higher confidence region is allowed for the adversarial examples to misclassify the correct label.

Another important property of adversarial example is its transferability. It is defined as the percentage of samples created for one model that can also fool another. It is reported in [24] that larger perturbation has better transferability. However, no study has been made about the transferability property of adversarial examples when the target in real scene is subjected to dynamic changes in viewing perspective and environmental condition such as orientations, distance and lighting conditions. Universal perturbation [36] shows only the existence of image-agnostic perturbations that can cause different objects to be misclassified with high probability. Unfortunately, it is not designed to cause misclassification of the same object under varying environmental conditions and viewing perspectives. Although these perturbations can generalize well across neural networks, they are quasi-imperceptible to human eyes only for misclassifying the target object with small sensing variations. The experiments with printed adversarial images in [9] demonstrated that the physical adversarial examples generated by FGSM and I-FGSM do not generalize well on dynamic environments. More powerful CW and EAD attacks [7], [8] with adjustable confidence, however, were not evaluated in [9].

Recently, the attack surface of adversarial examples has been extended from digital to physical [10], [11], [14] input by taking the viewing angles of the target into consideration. Sharif *et al.* [10] has demonstrated that it is possible for an attacker to evade face recognition systems by altering the eyeglass frame with printed adversarial perturbations. The perturbations to transfigure the eyeglass frame can be optimized on a set of inputs shown as follows to overcome small variations in expression and pose.

$$\argmin_{\delta} \sum_{x \in X} \text{softmaxloss}(J(x + \delta), c_t) \quad (5)$$

where *softmaxloss* is the score to measure the correctness of the classification and c_t is the attacker's target label.

To tolerate more dynamic visual shifts like viewing angles and distances, the perturbations can be created by robust physical perturbations (*RP2*) [11]:

$$\argmin_{\delta} \lambda \|M_x \delta\|_p + \mathbb{E}_{x_i \sim X^V} \text{loss}(J(x_i + M_x \delta), c_t) \quad (6)$$

where λ is a control parameter for the regularization of distortion, M_x is the perturbation mask to restrict the manipulation

area and X^V denotes the distribution that contains actual environmental variabilities and synthetic transformations. This algorithm will sample multiple instances x_i drawn from the distribution X^V , thus creating a valid perturbation for the distribution instead of a single instance.²

Besides patching an original object with crafted sticker, synthetical 3D adversarial examples can also survive varying viewpoints [14]. Such 3D samples can be produced based on a chosen distribution of transformations as follows:

$$\begin{aligned} & \argmax_{x'} \mathbb{E}_{t \sim T} [\log J(c_t | t(x'))] \\ & \text{subject to } \mathbb{E}_{t \sim T} [d(t(x'), t(x))] < r \end{aligned} \quad (7)$$

where $d(\cdot, \cdot)$ is a distance function to constrain the expected effective distance and r is the disturbance radius. The distribution T involves transformation functions t such as random rotation, translation, addition of noise and 3D rendering. The modeling of transformation on 3D rendering is captured from pre-built 3D models.

Both digital adversarial examples in a white-box setting and physical adversarial examples attached onto physical objects treat the DNN algorithm as an end-to-end process. So the perturbations are added onto the input sample instead of the internal process variables of the DNN. In order to fool deep learning classifier, it is also possible to move the sticker from the object to the lens of the camera [12]. But the disturbance is still imposed on the input. Even if the perturbations are created by considering the prediction loss and the hidden layer features loss [13], the attack is still limited by input-only manipulation. This constraint has severely undermined the success rate and robustness of both digital and physical adversarial examples. In addition, multiple pristine samples are required for physical adversarial algorithms. The indices of the summation operator $\sum_{x \in X}$ of (5), and the expected value operator, $\mathbb{E}_{x_i \sim X^V}$ of (6) and $\mathbb{E}_{t \sim T}$ of (7), express concisely the uncompromisable prerequisite of the quantity, besides the quality, of the input sampling for the generation of robust physical adversarial examples. The collected samples should also represent the complex distribution of the varying physical conditions.

C. Fault Injection Attacks on DNN Hardware

Directly perturbing an internal computation of a deep learning hardware can easily cause a classifier to output incorrect result. The internal process can be tampered by fault injection such as laser beam interference [16], memory collision [21] and rowhammering [22]. Pointing a focused laser beam to the exposed silicon die can skip part of the instructions of the ReLU function to introduce computation errors [16]. Model weight interpolation has also been studied. Such falsification can directly affect the MAC operations between the weight parameters and the input features. Single bias fault on the last fully-connected layer [17] and small deviation on multiple parameters of the layer [18] have been simulated to show

¹We refer the readers to [7] for more details on the selection of objective function.

²The terms, Non-Printability Score (NPS) and alignment function $T(\cdot)$, have been omitted in (6). NPS caters for the fabrication error and $T(\cdot)$ is used to align the perturbation on rotatable object view [11].

that they are possible to successfully deceive a DNN classifier. The weight parameter can also be flipped by writing opposite data to the same address of the dual-port random access memory (RAM) [21] and applying numerous rapid row activations to a dynamic RAM (DRAM) [22]. Nevertheless, attacks that cause permanent physical damage (e.g., wiping off chip package) or leave modification traces in memory can be easily detected and blocked by the device owner. Some hardware Trojans targeting memory controller [20] and activation function [19] have been suggested but these Trojan payloads are only emulated. One major limitation of hardware Trojan based attacks is that the Trojan circuit can only be secretively implanted during the hardware development process through malicious insiders or third parties.

A more practical inconspicuous attack has been successfully demonstrated on FPGA-based DNN accelerator recently [23]. It utilizes clock glitches [37] to inject temporal faults into the intermediate results of the convolutional layer and fully-connected layer. The glitches cause the PE to violate the timing constraint in (8) such that the convolution operations executed in those clock cycles are marred with errors.

$$T_{clk} \geq t_{pcq} + t_{pd} + t_{setup} + t_{skew} \quad (8)$$

where t_{pcq} is the worst case delay required to change the output of a flip flop from the arrival of the active clock's edge, t_{pd} is the maximum combinational circuit delay between two flip-flops, t_{setup} is the setup time of the flip-flop and t_{skew} is the uncertainty of the clock period.

The presence of glitch in the clock signal reduces the clock period T_{clk} or increases the clock skew t_{skew} . The setup time violation causes metastability when the asynchronous data and clock signals are combined to produce an indeterminate state at the flip-flop output. This will cause the erroneous intermediate results to be latched and propagated to the end of the inference stage. Once the glitch has subsided, correct timing is restored and the erroneous outputs are overridden by the new correct computation results thereafter. Hence the prediction errors only happen at a specific time of attack, which leave no trace for detection. The effectiveness of this attack has been demonstrated by injecting uniform glitches on all the layers, however, layer-wise attack has not been fully evaluated.

Although existing fault attacks [16], [21], [23] on endpoint device can subvert DNN output, none of these attacks has attempted to achieve robust misclassification nor has been evaluated against a diversity of visual variations such as different viewing angles, illuminations and distances. Comparing with using specifically designed digital and physical adversarial examples to steer a well-trained classifier to produce erroneous outputs, it is envisaged that this robust attack model is more likely to be unleashed by hardware-based attacks that can directly intervene the internal computations of machine learning system.

III. PROPOSED GLITCH INJECTION ATTACK

A. Attack Model

Whether an attack is based on perturbations created on the source or the process, the stealthy requirement must be

abided. For process tampering to remain undetectable during the attack or untraceable upon attacked, the frequency of disruptions and unavailability must be kept sufficiently low. As a result, model interpolation attack has been ruled out as it will leave footprints in memory for readback detection. Besides, attacks that cause physical damage, denial of service, device overheating or apparent performance degradation are also considered as “not stealthy”.

We assume that the adversary is targeting on real-world DNN accelerators. The attacker can cause the DNN's internal computations to be temporarily perturbed through non-invasive fault injection methods such as external IO injection [37] and software-based injection [38]. Traditional physical injection requires the attacker to have physical access to the device. This is made possible by the deployment of the DNN hardware at publicly accessible endpoint. Usually the edge AI product will have the AI accelerator chip mounted in a printed circuit board with other commercial off-the-shelf (COTS) chips and components. The board will be enclosed in a casing designed for aesthetic or utilitarian appeal. This casing is openable for the ease of testing and maintenance. The adversary can replace an adjacent COTS chip of known standard functionality on the circuit board by a malicious chip of the same functionality but embedded with the glitch injection circuitries. The fake chip has similar but multiplexed IO pins that makes it physically indistinguishable from the COTS chip it replaced. Alternatively, the adversary can also exploit the generic software-controlled fault injection methods reported recently on modern energy-efficient commodity devices equipped with DVFS. DVFS involves the interaction between hardware and software systems including hardware regulators and software drivers. To achieve cooperative energy management, the kernel driver will dynamically set the operating frequency and voltage based on the workload. This privilege to control the hardware regulators at a very fine granularity offers an opportunity to inject faults into the target circuit. It has been demonstrated that the control privilege of DVFS can be escalated by malicious kernel driver to inject oblivious faults into ARM SoC [38], [39], x86 processor [40]–[42] and GPU [43]. Given that DVFS is the predominant and widely used technique for hardware acceleration of DNN computations, timing errors can also be introduced into the clock signal of the PE array by sending malicious commands through the host processor without the physical access to the endpoint device. We assume that the attacker knows the general architecture of the deep learning accelerator. The attacker is also assumed to know the model structure but not the weight parameters of the DNN. This is a reasonable assumption as the structure of the DNN model running on the accelerator can be easily acquired by the memory and timing side channel information but not as easily for the weight parameters [44]. Due to the pipelined processing of MAC operations between adjacent convolutional layers of the DNN, an attacker can predict the execution time of each layer, and inject faults into the target layer(s) accordingly.

The attack model of adversarial examples requires the attacker to know when the target image is captured by the sensor in order to directly apply the image-specific perturbations onto the target image before it is presented to the model input.

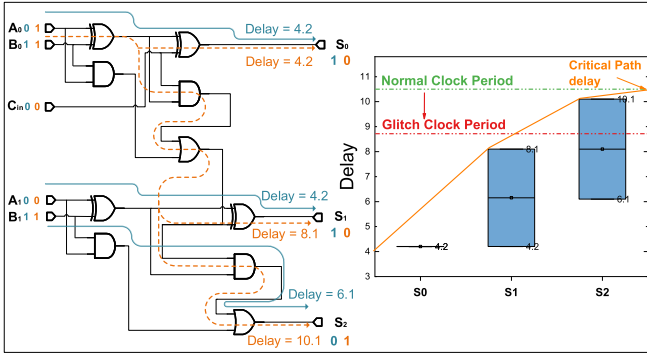


Fig. 2. Fault bias model of 2-bit ripple carry adder. Gate delays are normalized by 2-input NAND gate delay, where $D_{nand} = 1$, $D_{and} = 2.2$, $D_{or} = 1.8$ and $D_{xor} = 2.1$.

In contrast, our attack is more practical as it only requires the attacker to know when the target object is presented to the DNN model without the need to tamper with the target image or the object. Therefore, the attack is less hindered by the constraints of object viewpoint and scene variations. We further assume that the attacker has only one sample of the object in a fixed scene to derive the attack vector of a target object imaged with different viewing angles, background illuminations and distances of the object from the camera. This restriction is more pragmatic and significantly more stringent than that for physical adversarial examples.

Based on those constraints, two modes of penetration, *covert* mode and *robust* mode, are considered in our attack. In the covert mode, the injected signal perturbs a minimal number of internal operations from input to inference. In the robust mode, the attack vector maximizes its adaptivity to a larger variance of object and scene attributes.

B. Clock Glitch Injection

Following (8), clock glitch reduces the clock period and can cause timing violation of arithmetic datapaths in the affected cycles. This timing violation introduces biased faults on the output bits where higher order bits are more likely to be flipped in the vicious clock cycle [45]. Fig. 2 illustrates the phenomenon of fault bias by an example of a 2-bit ripple carry adder. The blue solid line delay path corresponds to the input operands $A = 00$ and $B = 11$ with $C_{in} = 0$, whereas the orange dotted line path corresponds to the input operands $A = 01$ and $B = 11$ with $C_{in} = 0$. The clock period is bounded by the critical path delay. The time taken for all output bits to stabilize at their final values depends on the inputs. Due to the inevitable carry propagation, higher order output bits (e.g., S_2) take longer time to settle than the lower order bits. By injecting glitches into the clock signal, the output latches are more likely to latch on incorrect higher order bits [46]. Similar effect was demonstrated in Plundervolt [40], where a DVFS fault can be controlled by the software on Intel processor to cause flipping of contiguous bits or the most-significant bit and sign-extended bits of a hardware multiplier output. This biased fault is favorable to glitch attacks as a glitch injected in the middle of a clock

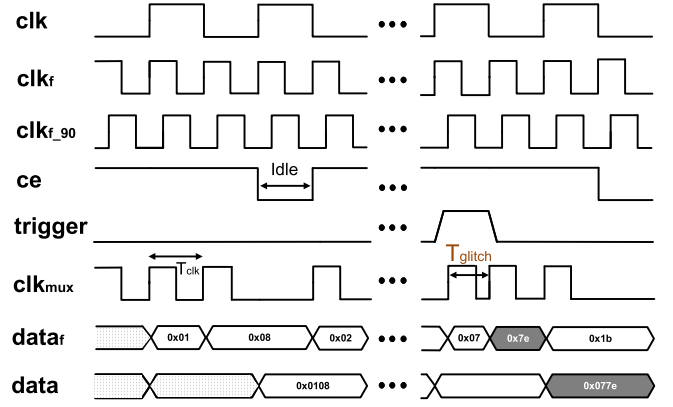


Fig. 3. Timing waveforms of glitch injection attack on DNN accelerator. T_{clk} is the nominal clock period and T_{glitch} is the reduced clock period with glitch injection. The gray shaded blocks indicate erroneous data and the unshaded blocks are correct intermediate results.

cycle, without having to be precisely positioned, is possible to cause a large error on the computation result.

Similar to [23], we make use of the separate clock domain of the PE array for glitch injection. Perturbing the clock signal affects only the calculation of the PE array but not other functional modules. The errors introduced into the computation results of the current glitch-infected cycle will feed forward to the inference stage to affect the classification result of the current frame. The same computation of the next input data will not be affected once the glitch has subsided. Hence the attack is stealthy. In addition, we also exploit the gated clock in typical modern DNN accelerator depicted in Fig. 1 for fine-grain glitch attack. The computational workload and activity exposed by the enable signal of the gated clock can be used to identify the operation schedule.

Fig. 3 shows the timing waveforms of several salient signals involved in the glitch injection attack on deep learning accelerator. The master clock signal clk is applied to the general logic such as the configuration unit, data controller, global buffer and SIMD engine of the accelerator. A fast reference clock clk_f is derived from clk to accelerate the calculations of the PE array. The 90° phase shifted clock signal clk_{f_90} is generated by shifting clk_f by 90°. A pulse $trigger$ is generated to produce a multiplexed signal clk_{mux} of clk_{f_90} and clk_f . By connecting clk_f and clk_{f_90} to the data inputs and $trigger$ to the select input of a 2-to-1 multiplexer, the multiplexer outputs $clk_{mux} = clk_f$ if $trigger = 0$ and $clk_{mux} = clk_{f_90}$ if $trigger = 1$. With the output of the multiplexer connected to the clock input pin of the DNN accelerator, glitches can be selectively injected into the clock signal of the DNN accelerator by controlling the timer output $trigger$. The clock enable signal ce is used by the DNN accelerator to inhibit the input clock to the PE array during the idle interval to avoid spurious switching. A glitch cycle T_{glitch} refers to one cycle of clk_{mux} when $trigger$ is asserted. By driving the PE array with clk_{mux} instead of clk_f , erroneous outputs will be produced by some or all PEs in the T_{glitch} cycles due to setup time violation of (8). The gray shaded blocks in Fig. 3 represent the erroneous data that will be

Algorithm 1 Extraction of Prospective Attack Patterns

Input: Model(J), Layer list(L), Glitch Intensity list(G), Image(I), Correct label (c_o)

Output: Prospective pattern list (V)

// initialize candidate pattern list $V \leftarrow \{\}$

// initialize index $i \leftarrow 0$

```

1: for each  $g \in G$  do
2:   for each  $l \in L$  do
3:      $v \leftarrow \text{trigger}(l, g)$ 
4:      $c_v \leftarrow \text{class}(J, I, v)$ 
5:     if  $c_v \neq c_o$  then
6:        $i \leftarrow i + 1$ 
7:        $V[i] \leftarrow v$ 
8:     end if
9:   end for
10: end for
11: return  $V$ 

```

propagated from the local data bus $data_f$ of the PEs to the global data bus $data$. The errors incurred in any operations during T_{glitch} will propagate and accumulate in succeeding computations until the final prediction of the current input is completed. The rate of the attack depends on the number of T_{glitch} cycles. For ease of exposition, the ratio of the number of T_{glitch} cycles to the total number of normal clk_f cycles during an attack, expressed in percentage, is referred to as glitch intensity. The glitch intensity can be controlled by varying the frequency and/or duration of the *trigger* pulses.

C. Attack Patterns

An attack pattern is a timing pattern of the *trigger* signal generated by programming the frequency and duration of the arbitrary pulse generator output. Different attack patterns, even with the same glitch intensity, will induce different numbers and magnitudes of errors into different convolutional layers of the DNN computations in the PE array. Taking into account the secrecy and efficacy of an attack, two attack patterns, namely least glitch attack (LGA) and sensitive layer attack (SLA) patterns, are sought. LGA aims to minimize the glitch intensity of an attack pattern to misclassify a target under variational orientation and scene with reasonably good success rate. On the other hand, SLA aims to find an attack pattern that maximizes the misclassification rate of a target in varying physical conditions at the expense of higher glitch intensity.

A profiling procedure is used to extract a set of prospective attack patterns. Algorithm 1 describes the search procedure for the prospective attack patterns $\text{trigger}(l, g)$, where l is a selected layer and g is a selected glitch intensity for layer l . Given a hardware executable model J , a list of the model layers L , a set of glitch intensities G , the correct label c_o and a fixed scene image I of the target object, the search can be performed with layer-wise glitch injection of different intensities in two iterations. The list L also contains the start and end cycle numbers of each layer $l \in L$. In our experiment, $G(\%) = \{0.01, 0.02, 0.033, 0.1, 0.2, 0.33, 1, 2, 3.3, 9, 33\}$ with three different glitch intensities per decade from 0.01%

Algorithm 2 LGA Pattern Search

Input: V , Active cycles of each layer (E)

Output: LGA pattern (v_{lga})

// initialize $H \leftarrow \{\}$

```

1: for each  $v \in V$  do
2:    $g \leftarrow \text{glitch\_intensity}(v)$ 
3:    $e \leftarrow \text{num\_cycles}(E, \text{layer}(v))$ 
4:    $h \leftarrow e \times g$ 
5:    $H[v] \leftarrow h$ 
6: end for
7:  $\text{index} \leftarrow \text{argmin } H[i]$ 
8:  $v_{lga} \leftarrow V[\text{index}]$ 
9: return  $v_{lga}$ 

```

to 33%. The function $\text{class}(J, I, v)$ returns the classification result of input I by model J with clk_{mux} selected by v . The prospective pattern list V is obtained by comparing the correct label c_o against the label c_v predicted with each attack pattern v .

Searching for LGA patterns is similar to the gray-box attack procedure in [23], except that gating enable signal is exploited to eliminate redundant glitches and prevent any power monitoring circuit from detecting spurious MAC activities in the PE array during its idle time. This improvement can cut down about 50% of interferences during the idle period of the PE array. The set of active execution cycles of each layer (E) can be derived from the layer-wise execution time and the status of the gating enable signal. Algorithm 2 shows the LGA pattern search routine. The functions $\text{glitch_intensity}(v)$ and $\text{layer}(v)$ return the glitch intensity and the layer number of an attack pattern v , respectively. The function $\text{num_cycles}(E, l)$ extracts the number of active clock cycles in layer l from E . The number of glitch cycles introduced by an attack pattern v is given by the product of its glitch intensity g and the number of active clock cycles e of its target layer. Algorithm 2 returns the attack pattern v_{lga} with the minimum number of glitch cycles among the prospective patterns generated by Algorithm 1.

Different layers of a DNN are responsible for capturing different features. Some layers capture more general features shared by multiple objects, and others are more discriminative and can learn the exclusive features of specific object. For example, a discriminative layer can be more responsive to the car wheel due to the structure of circle and line conjunctions whereas another discriminative layer may respond more actively to a dog by the shape of eye and noise [47]. For a well-trained DNN, different objects will produce their respective most discriminative feature maps on different layers. If the stimulus-specific feature maps of an object in such layer are subverted by computational errors, then the same object is more likely to be misclassified invariantly regardless of its viewing point and the environmental noise. Our SLA pattern is extracted based on the correlation between the layer-wise response and the object label. Algorithm 3 describes the procedure of our SLA pattern search. The list N is used to store the number of prospective attack patterns $v \in V$ that can

Algorithm 3 SLA Pattern Search**Input:** V, L **Output:** SLA pattern (v_{sla})

```

// initialize  $N \leftarrow \{\}, M \leftarrow \{\}$ 
1: for each  $l \in L$  do
2:    $N[l] \leftarrow 0$ 
3:    $M[l] \leftarrow 0$ 
4:   for each  $v \in V$  do
5:     if  $l = \text{layer}(v)$  then
6:        $N[l] \leftarrow N[l] + 1$ 
7:        $M[l] \leftarrow \max(M[l], \text{glitch\_intensity}(v))$ 
8:     end if
9:   end for
10: end for
11:  $v_{sla} \leftarrow \text{trigger}(\arg\max_l N[l], M[\arg\max_l N[l]])$ 
12: return  $v_{sla}$ 

```

inject glitches into each layer $l \in L$. The list M is used to store the maximum prospective glitch intensity for each layer $l \in L$. The sensitivity of a layer to misclassify a given target is measured by its number of prospective attack patterns on that layer. The layer that has more prospective attack patterns for a particular input stimulus implies that it is more vulnerable to misclassification by computational errors. Therefore, SLA pattern targets the most sensitive layer l that has the maximum $N[l]$ value. To increase the attack robustness, the maximum glitch intensity for the most sensitive layer ($M[l]$) is chosen.

IV. EXPERIMENTAL RESULTS

In this section, we first introduce the test dataset, DNN models and experimental setups used for the evaluation of the adversarial examples and the proposed glitch injection attacks. Then, the results of physical adversarial examples, LGA and SLA based attacks are presented, discussed and compared.

A. Dataset and DNN Models

The publicly available Amsterdam library of object images (ALOI) [29] is used as our input dataset. It has a large collection of different objects under varying imaging conditions. Each object is recorded under 72 viewing angles,³ 24 lighting directions⁴ and 12 color illuminations.⁵ 118 objects from ALOI dataset, representing 25 ImageNet categories,⁶ are selected. Considering the realistic application scenarios, we select 61 environmental conditions including 25 viewing angles (-60° to 60°) with all cases of illumination directions and color temperatures of these objects. In total, 7198 images consisting of 118 objects with 61 different viewpoints are used

³Covering $0^\circ - 355^\circ$ by rotating the object in the plane at a step resolution of 5° .

⁴Combination of 5 lamps and 3 cameras.

⁵By changing voltage of the lamp for color temperature from 2175 to 3075K.

⁶Including acorn, analog clock, basketball, book, bowl, candle, cellphone, coffee mug, conch, digital clock, drum, fire truck, ladybird, lighter, perfume, pill bottle, police wagon, pot, racing car, shell, shoe, sock, teapot, teddy bear and wallet.

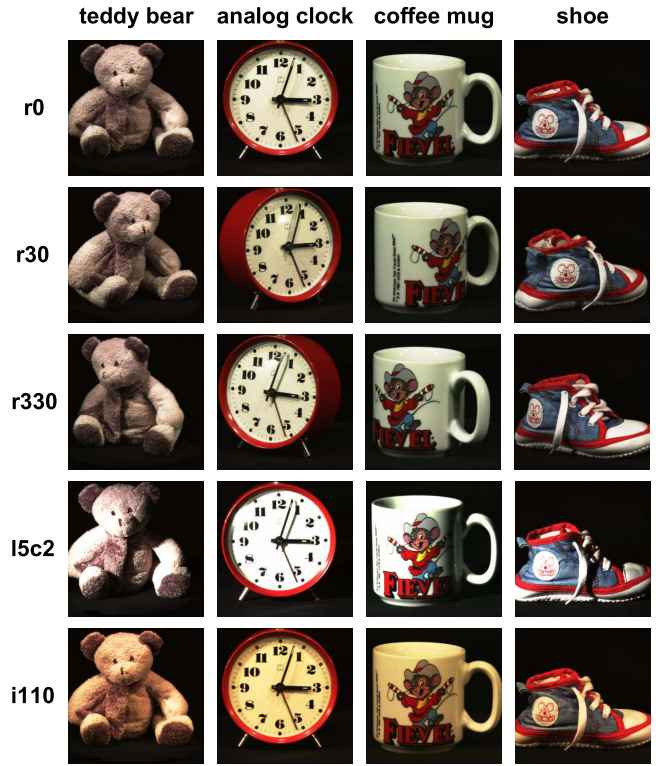


Fig. 4. Selected samples from ALOI dataset representing 5 out of 61 imaging conditions including 0° (r0), 30° (r30), -30° (r330) rotations, lighting from right front (l5c2) and color temperature at 2175K (i110).

for our evaluations. A sampling of some images are shown in Fig. 4.

As for the deep learning classifiers, several representative network architectures from each of the four well-known models, i.e., ResNet50 [25], Inceptionv4 [26], MobileNetv2 [27] and VGG16 [28], are targeted. Both the adversarial examples and our proposed hardware attacks are conducted on these selected models using the selected images.

B. Design of Experiments

Our glitch attack is mounted on the FPGA implementation of the aforementioned DNN architectures. A Xilinx ZCU102 evaluation board, which features a ZYNQ UltraScale+ (XCZU9EG-2FFVB1156) MPSoC device, is used to emulate the edge device. This SoC consists of a quad-core ARM CortexTM-A53 processor, 600K system logic cells and 2520 dedicated DSP slices. The block diagram of our setup is depicted in Fig. 5. Hardware acceleration of DNN computations is achieved by the deep learning processor unit (DPU) provided by Xilinx [34]. DPU is a soft IP which can be configured by different parameters, such as different utilization rates of DSP slices, look-up table (LUT) and on chip memory, to suit a series of FPGA devices with different available programmable fabrics. We follow the configuration in [34] by setting the maximum parallelism of the convolution architecture (B4096) and high DSP usage. This allows the DSP units to be utilized for acceleration

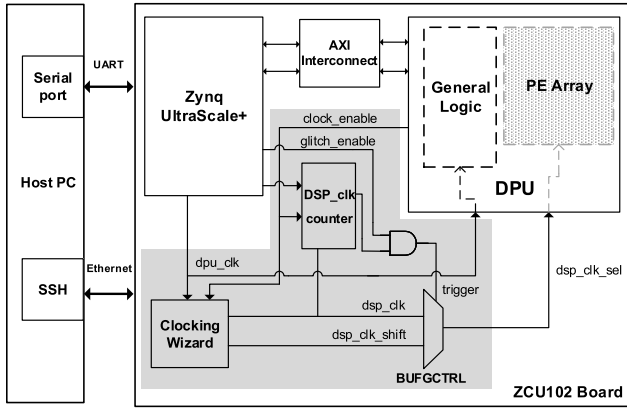


Fig. 5. Experimental setup of glitch attacks on Xilinx ZCU102 platform. The additional fabrics for glitch injection include a Clocking Wizard, a DSP clock counter, an AND gate and a BUFGCTRL. Their IO interfaces to the original circuit are demarcated by the gray shaded block.

of both multiplications and accumulations, which are the bottleneck arithmetic operations of DNN datapaths.

Similar to typical deep learning accelerators, DPU provides a separate clock domain for array of MAC operations (viz. DSP slices). The glitch injection is accomplished through four peripherals demarcated by the boundaries of the gray shaded blocks in Fig. 5. The fast dsp_clk clock and its 90° phase shifted clock dsp_clk_shift are generated from the normal DPU clock dpu_clk by the Clocking Wizard. A global clock buffer with multiplexer (BUFGCTRL) is instantiated to select either dsp_clk or dsp_clk_shift to dsp_clk_sel of the PE array. The $clock_enable$ signal is feeded back from the DPU to reduce its power consumption when the computation units are idle. DSP clock counter will base on the $clock_enable$ signal and the clock cycle-accurate timing information received from the ARM CortexTM-A53 processor to time and toggle the $trigger$ pulses on or off. The $glitch_enable$ signal is used to start and stop the glitch attack. Configuration signals including glitch intensity, $glitch_enable$ and the timing of the T_{glitch} cycles are sent from the host PC to the Extended Multiplexed IO (EMIO) pins of the ARM CortexTM-A53 processor (Zynq UltraScale+ block) to produce the attack patterns. These EMIOs can be read or written through the OS kernel. The host PC can control and transfer data to the FPGA board via the UART and Ethernet connections. In our design, the DPU runs at 100MHz and the DSP runs at 200MHz. Hardware executable DNN models are compiled by the Deep Neural Network Development Kit (DNNDK) [48] toolchain. The Deep Compression Tool (DECENT) of DNNDK converts the 32-bit floating point weights and activation into 8-bit integers to reduce the computing complexity without losing the prediction accuracy of the DNN models.

Besides our proposed hardware attack, other attacks based on state-of-the-art adversarial examples are also conducted on the same 118 objects with 61 different imaging conditions. Specifically, FGSM and I-FGSM attacks are conducted based on the setting of [49], and CW and EADL1 attacks are conducted with the framework of [24]. These experiments are performed on the NVIDIA GTX1080 GPU platforms. Our evaluation is far more comprehensive than the earlier

TABLE I
SUCCESS RATES (%) OF MISCLASSIFYING ALOI IMAGES BY DIFFERENT ATTACK METHODS ON FOUR DNN MODELS

Attack	Shift	Inceptionv4	MobileNetv2	ResNet50	VGG16
CW	angle	13.4	24.8	12.4	6.3
	light	25.6	37.5	48.4	6.2
	color	10.7	22.4	16.2	5.4
	Overall	17.5	28.9	26.6	6
EADL1	angle	8.0	7.5	13.3	6.9
	light	5.1	8.7	15.4	3.7
	color	8.2	8.7	11.2	10.3
	Overall	6.9	8.2	13.6	6.5
FGSM	angle	13.4	15.6	15.3	15.1
	light	13.6	30.8	23.0	21.5
	color	32.5	84.7	64.4	73.7
	Overall	17.4	36.2	28.7	30.2
I-FGSM	angle	10.9	38.5	14.3	26.7
	light	8.6	45.8	21.8	35.7
	color	46.8	96.0	82.0	92.5
	Overall	17.3	53.6	31.7	44.4
LGA	angle	58	70.2	52.8	59.8
	light	60.6	73.6	56.3	59.1
	color	59.1	71.3	57.5	64.9
	Overall	59.2	71.7	55.1	60.7
SLA_Min	angle	64.7	67.7	58.1	65.2
	light	64.7	70.1	63.1	65.9
	color	66.2	71.8	63.1	61.1
	Overall	65	69.5	61.1	66.6
SLA_Max	angle	98.5	95.9	95.1	95.8
	light	98.4	97.5	95.9	96.7
	color	99	94.4	98.6	97.2
	Overall	98.6	96.2	96.2	96.5

adversarial attack experiments on DNN models for traffic sign detection with changing scene scenarios [9]. The latter used only one stop sign target in 30 different views to mount the adversarial examples generated by the earliest Limited-memory Broyden-Fletcher-Goldfarb-Shanno (L-BFGS) [4], FGSM and I-FGSM.

The 0° (r_0) samples of correctly classified objects on each DNN model are singled out as the reference for the attack outcome analysis by Algorithm 1 and adversarial example attacks. The other viewpoints of the same objects are used for the evaluation of attack. Only those original untampered images that can be correctly classified will be attacked. Original variational viewpoint images that cannot be correctly recognized by the classifier are excluded. The success rate of an attack is calculated based on the successful misclassification of those object images that can otherwise be correctly classified without the attack.

C. Results of Adversarial Example Attacks

In this experiment, attacks by four different methods of adversarial examples are evaluated. For FGSM and I-FGSM, the perturbations are created based on the search for the minimum valid ϵ from 1 to 32. For CW and EADL1, the confidence parameter is set to $\kappa = 0$, which is the default setting in [7], [8]. The results broken down into different viewing angles, lighting directions and illumination colors are summarized in the first four main rows of Table I. Although adversarial examples of CW are quasi-imperceptible and more powerful for view-invariant targets, the algorithm has limited generalization capability to dynamic environmental variations. Its success rate is below 29%. The results are particularly poor on varying angles and color temperatures. Another

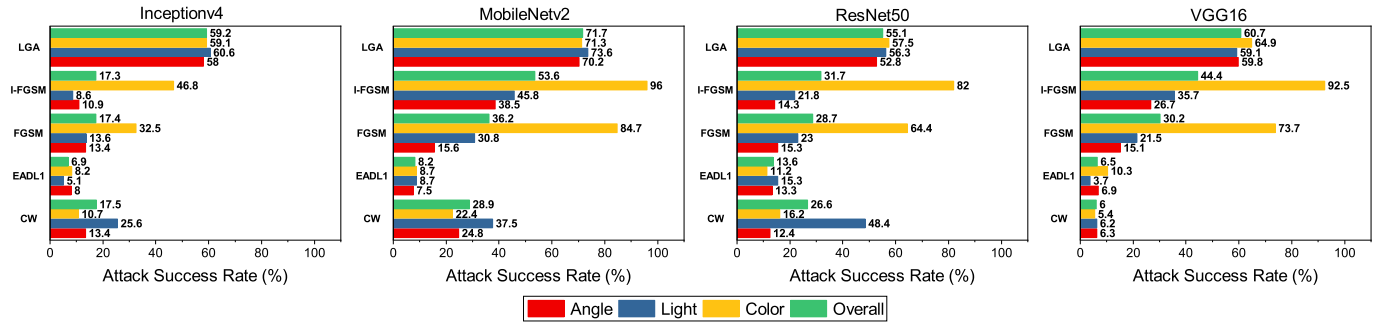


Fig. 6. Comparison of LGA with state-of-the-art adversarial attacks on different DNN models for ALOI images. The attack success rates on different viewing angles, light directions, illumination color temperatures and the overall success rate for all variation cases are shown by the red, blue, yellow and green bars, respectively.

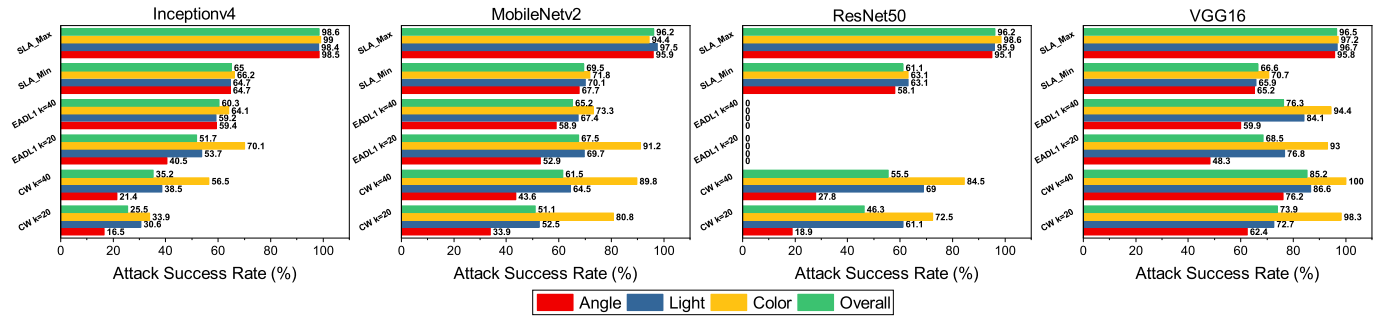


Fig. 7. Comparison of SLA with high-confidence CW and EADL1 attacks on different DNN models for ALOI images. The attack success rates on different viewing angles, light directions, illumination color temperatures and the overall success rate for all variation cases are shown by the red, blue, yellow and green bars, respectively.

quasi-imperceptible adversarial attack algorithm EADL1 also does not generalize well on multiple viewpoints. Its success rate is less than 14% on all the evaluated DNN models. On the contrary, FGSM and I-FGSM have higher success rates especially for targets with different illumination colors. However, their attack success rates on high-accuracy DNN models like ResNet50 and Inceptionv4, even with perceptible perturbations, still fall below 32%.

This evaluation reveals that adversarial examples have very limited robustness against subtle context variations. Perceptibility minimized adversarial examples, such as CW and EADL1, can neither transfer properly on multiple models [24] nor generalize well on varying viewpoints. FGSM and I-FGSM can generalize only slightly better by lifting the imperceptibility requirement. Based on visual imperceptibility of adversarial examples, it is interesting to note that the relative attack robustness of these algorithms against elusive object-scene context changes follows a similar trend as their transferability. When $\kappa = 0$, the transferability is $\text{FGSM} > \text{I-FGSM} > \text{EADL1} > \text{CW}$ [24].

D. Results of Proposed Least Glitch Attack

The bar chart in Fig. 6 compares the success rates of the four models attacked by our proposed LGA against the four adversarial examples. For each attack method, the misclassification rates of objects with different viewing angles, light directions and illumination colors are represented in red, blue and yellow, respectively. The overall success rate of all evaluated images is shown by the green bar. The numerical success rate of each bar of LGA can be found in the row label “LGA” in Table I.

TABLE II

AVERAGE GLITCHES PER IMAGE REQUIRED BY LGA ON DIFFERENT MODELS

	Inceptionv4	MobileNetv2	ResNet50	VGG16
Total #Cycles	15487756	1115101	4672551	17183856
Glitches	322.9	2.38	964.12	3333.89
Percentage (%)	2.1×10^{-3}	2.1×10^{-4}	0.021	0.019

The proposed LGA outperforms all adversarial examples, with overall attack success rates ranging from 55.1% for Inceptionv4 to 71.7% for MobileNetv2. The minimum winning margin is 16.3% over I-FGSM for VGG16 and the maximum winning margin is 63.5% over EADL1 for MobileNetv2. Specifically, LGA achieves consistently higher misclassification rates than all adversarial examples by at least 31.7% and 7.9% for varying viewing angles and lighting directions, respectively. The attack success rates of FGSM and I-FGSM are higher for different color temperatures. The most likely reason is the perturbations generated by FGSM and I-FGSM are more sensitive to the changes in high-level features such as structures and contours but are less dependent on the color features of the input image. Nevertheless, the overall success rate of LGA still exceeds those of FGSM and I-FGSM for all DNN models by at least 23.4% (ResNet50) and 16.3% (VGG16), respectively.

Table II summarizes the demand on average glitches per image of LGA. This number of glitches is averaged over all evaluated images for each deep learning model. Total #cycles refers to the number of DSP clock cycles required for an

inference, which can be monitored by the clock counter.⁷ It is found that a very small amount of glitches is sufficient to launch LGA. Specifically, two glitches in a million clock cycles are adequate to attack MobileNetv2. Two injections in ten thousand computations are also sufficient to attack ResNet50 and VGG16. A low number of glitches needs few deliberate changes on the hardware regulator, and evokes less spurious signal transitions on the data bus and control signals. The insignificant fraction of malicious signal transitions can be easily buried away in the noisy operating environment, and is less likely to be detected as abnormality by typical signal regulator and power density monitor.

E. Results of Proposed Sensitive Layer Attack

The attack pattern v_{sla} (SLA_Max) has the maximum glitch intensity on the most responsive layer of each target. To contrast SLA_Max, attack pattern (SLA_Min) with the minimum glitch intensity on the most responsive layer l of each target is also extracted by initializing $M[l]$ to 33% and changing $\max()$ to $\min()$ in Line 7 of Algorithm 3. For fairness of comparison, the adversarial examples of CW and EADL1 attacks are generated with high confidence settings ($\kappa = 20$ and $\kappa = 40$). $\kappa = 40$ is the optimal value reported in [7] to yield the strongest adversarial examples for these methods. Increasing κ beyond this threshold tends to degrade instead of improving the attack performance [7], [24].

The attack results are compared in Fig. 7. Significantly higher attack success rates are observed on SLA_Max. The overall attack success rates on all models are higher than 96.2%. In addition, its success rates for attacking objects with varying viewing angles are higher than all the high confidence methods of adversarial examples by 19.6% to 82%. For different lighting conditions, SLA_Max outperforms other attacks by 10.1% to 67.8%. The most robust model, i.e., VGG16, under the more secretive CW and EADL1 in Fig. 6, has become vulnerable under the same attacks with greater κ . With relaxed attack imperceptibility, VGG16 becomes almost equally vulnerable to all attacks for objects with different color temperatures. It is worth noting that even SLA_Min is capable of achieving better success rates than high-confidence adversarial examples on the majority of DNN models, including Inceptionv4, MobileNetv2 and ResNet50. Increasing κ can improve the success rates of CW and EADL1 attacks on most network models in all sensing conditions except ResNet50. It is found that EADL1 with $\kappa = 20$ and $\kappa = 40$ failed to produce valid perturbations to misclassify any r0 sample. Hence, their misclassification rates for ResNet50 are zero for all sensing conditions. Similar effect has been reported in [8], [24], as valid adversarial examples of certain networks are harder to find by minimizing the objective function in (4) with larger κ .

⁷We find that each inference may consume slightly different number of DSP clocks even for same input image. The difference is less than 0.01% of the total #cycles. This phenomenon may be attributed to complex hardware execution flow such as cache hit/miss or unbalanced data stream. Reported total #cycles in Table II are the averaged number of clock cycles over multiple rounds of the same inference.

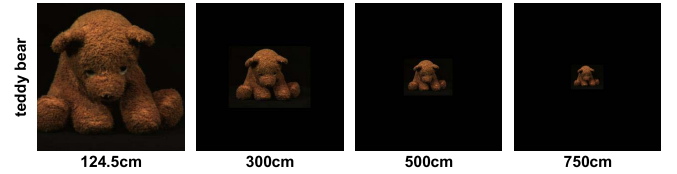


Fig. 8. Simulated images of a teddy bear captured at 300cm, 500cm and 750cm.

TABLE III
ATTACK SUCCESS RATES (%) OF PROPOSED ATTACKS ON OBJECTS CAPTURED AT DIFFERENT DISTANCES

	Inceptionv4	MobileNetv2	ResNet50	VGG16
LGA	75.9	61.1	80.5	75
SLA_Min	75.9	72.2	92.7	75
SLA_Max	100	100	100	100

TABLE IV
COMPARISON BETWEEN PROPOSED ATTACKS AND PHYSICAL ADVERSARIAL EXAMPLES ON SENSING CONDITIONS, TAMPER ORIGIN, NUMBER OF REFERENCE SAMPLES AND METHODOLOGY

	Ours	Eykholt's [11]	Zhao's [13]	Athalye's [14]
Angle	-60° - 60°	$\leq 60^\circ$	$\leq 60^\circ$	walk around
Illuminant	2175-3075K	lab/field	sunny/cloudy	walk around
Direction*	24	–	1-3 pm	walk around
Distance	1.25-12.5m	1.52-12.2m	5-25m	walk around
Origin	Process	Source	Source	Source
#Samples	One	Multiple	Multiple	Multiple
Method	Glitch	Sticker	Sticker	3D Printing

* Different light directions

F. Comparison With Physical Adversarial Examples

Object distance from camera has been considered in physical adversarial examples. Unfortunately, the ALOI dataset does not include images of object taken from different distances. To compare our glitch injection attack with physical adversarial examples, the predominant perspective changes due to different camera-object distances are simulated by the inverse linear relationship between the object size and distance. The recording distance of the images in ALOI is 124.5cm [29]. This distance is used as a baseline to simulate images of the same object captured at other distances, including 300cm, 500cm, 750cm and 1250cm. Several simulated images of a teddy bear at different distances are shown in Fig. 8. Table III summarizes the results of LGA, SLA_Min and SLA_Max mounted on the object with changing distances. All models can be attacked by LGA with a success rate higher than 60%. More importantly, all models can be successfully attacked by SLA_Max with 100% misclassification rate.

Existing physical adversarial examples create perturbation based on a set of samples, and evaluate their effectiveness in multiple scenes using self-collected data [11], [13], [14]. Table IV shows the major differences between our approach and cutting-edge physical adversarial examples in terms of sensing conditions, target manipulation, number of reference

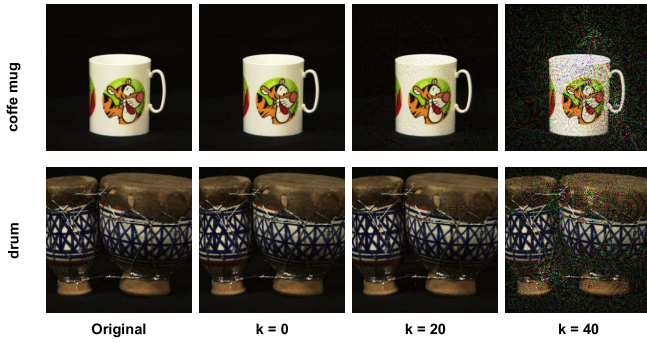


Fig. 9. Adversarial examples created by EADL1 with $\kappa = 0, 20, 40$ on Inceptionv4.

target samples and methodologies of attacks. In general, our evaluations are more precise, systematic and comprehensive in terms of attack scenarios, especially on variations in target viewing angles and illuminations. Our attack has been evaluated on 25 orientations ranging from -60° to 60° , whereas the attacks in [11] and [13] are reported with only 5 specific angles (i.e. $0^\circ, 15^\circ, 30^\circ, 45^\circ$ and 60°) and 4 regions (i.e. $0^\circ, 30^\circ, 45^\circ$ and 60°), respectively. Our approach has been evaluated on different color temperatures from 2175 to 3075K and 24 combinations of light directions. In comparison, Eykholt *et al.* [11] evaluated their attack on only two general types of surrounding illuminations, namely indoor (lab) and outdoor (field), without specifying the light directions. Zhao *et al.* [13] investigated their attack on sunny and cloudy weather as well as sunlight variations during 1-3pm. They also evaluated their attack on the target captured at longer distance of up to 25m. The object size in the captured image is dependent on the camera's distance and focus length. For the ALOI images captured by Sony DXC390P 3CCD camera (with Computar 12.5-75mm, 1:1.2 lens set to $f = 5.6\text{mm}$ with zoom = 48 mm or 15 mm), our simulation shows that the object becomes too small to be recognized when the distance is increased beyond 12.5m. The attack method [14] was evaluated by walking two 3D printed adversarial examples around and moving them up and down without specifying other imaging conditions. Our proposed attack requires only one image sample of the target to attack the same target in other conditions. In contrast, other physical adversarial examples generated by (6) and (7) require multiple image samples of the target captured in different conditions. The key difference that makes our attack more robust without requiring multiple reference samples is that our method tampers the processing of DNN whereas the physical adversarial attacks rely merely on manipulating the input source to tackle the sensing variations.

V. REFLECTION

A. Attack Robustness and Tamper Origin

Hardware fault attack differs from adversarial examples in the origin and constraints of perturbation propagation. Manipulation on the input image or physical target requires the perturbations transmitted from the source to take effect at the final classifier after traveling through all the layers. Although

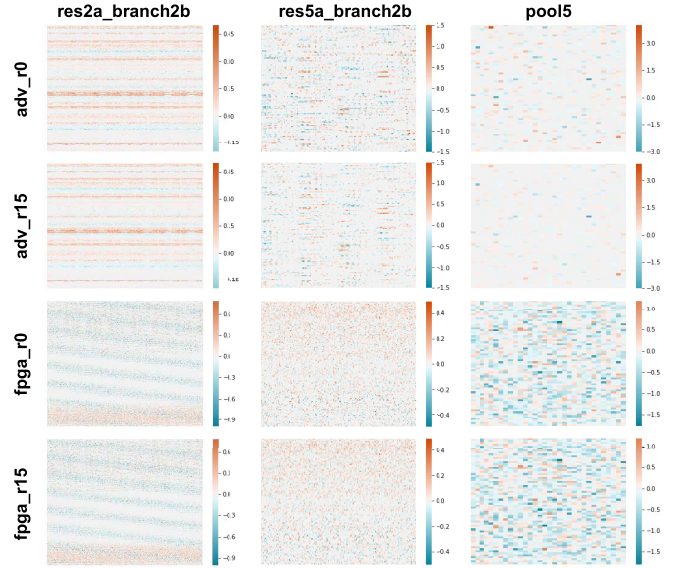


Fig. 10. Deviation of intermediate features corrupted by adversarial examples and SLA_min with 1% glitch intensity on ResNet50.

it is possible to misclassify a target with small imperceptible perturbations, the attack success rate drops tremendously when the sensing condition of the target changes dynamically. Increasing the perturbations will inevitably reduce the attack imperceptibility, with limited improvement on attack robustness as shown in our experiments. Fig. 9 shows some adversarial examples generated by applying EADL1 on Inceptionv4 with different confidences. Stealthy EADL1 attack ($\kappa = 0$) can produce a perturbed image that is visually indistinguishable from the original image. However, it has limited success in misclassifying the same target with changing environments. Increasing κ to 40 can increase its attack success rate, but the perturbations are conspicuous and hard to be ignored by human observers. Similarly, more robust physical adversarial examples require high contrast stickers [11] or nonobjective shapes [13]. It is conjectured that large input distortion is required to counteract the inherent denoising property of neural networks. For example, the pooling layer can pool out the critical features and exclude those non-critical ones. Consequently, a small perturbation that is effective on one scene may be filtered out by the internal layers as noise in some other scenes. The constraint of manipulating presented input limits the adversarial examples from interpolating intermediate features.

As opposed to tampering the input or physical object, hardware attacks can directly induce large errors into an internal layer through fault injection. This extra degree of freedom unlocks their versatility for variational attacks. Injecting faults into a stimulus-sensitive layer can bypass strong noise removal from preceding layers. Such small fault can easily escalate to large deviations of internal features without changing the target appearance at the input, hence there is no visual perceptibility issue. Fig. 10 shows the deviations of intermediate features perturbed by adversarial examples and glitches injection. The first two rows are features obtained from original r0 and

r15 samples perturbed by adversarial distortions, while the third and fourth rows are the features obtained from the same original samples with clock glitches injected into computation cycles of res2a_branch2b layer. The deviations are illustrated by heatmap, where 0 (blank block) data point means that the feature point has no difference with and without perturbation. For attacks by adversarial examples, the perturbations are gradually receded as they propagate from shallower layer to deeper layer, as evinced by the changing of non-blank sections from dense to sparse in Fig. 10. Specifically, the heatmap of adv_r15 on pool5 layer has much less biased data compared with adv_r0 sample, which accounts for the unsuccessful attack on r15 sample. For glitch attacks, the injection on res2a_branch2b layer is effective for the r0 sample. By applying the same attack pattern on r15 sample, similar distortions can be observed on the selected layer shown in Fig. 10. The injected faults can propagate to deeper layer, such as pool5 layer, and account for a successful attack.

Besides attacking an object under a variety of viewing conditions, it is interesting to explore the transferability of applying a successful attack pattern on another object. A preliminary evaluation of the attack pattern transferability has been performed on ResNet50 for four objects in ALOI dataset, which are analog clock, soup bowl, coffee mug and teddy bear. Apparent reduction in misclassification rates were experienced by applying both stealthy and robust attack patterns on non-target objects. The attack pattern transferability was not always symmetric between object pairs. The conditions to achieve a good transferability of an attack pattern from one object to another is unclear. A more systematic and comprehensive study of the transferability of glitch injection attack is a good topic for future research.

B. Potential Defense

Several preprocessing techniques such as discrete wavelet transform [50] and adaptive noise reduction [51] can mitigate success rate of adversarial attacks on DNN model. These techniques are designed to distill the input sample but not the internal process targeted by the versatile glitch injection attacks. To increase the tolerance of DNN accelerator against hardware-based or process-oriented attacks, the fortification has to be built into the DNN model and hardware circuit. For example, the error amplification effect during inference can be reduced by defensive quantization [52], which applies constraint on Lipschitz constant during the model training phase. This method is originally intended to improve the reliability of an accelerated model implementation that has been quantized below 5 bits. Increasing the number of neurons in the hidden layer and restricting the parameters within a fixed boundary can also improve the robustness of DNN model against the fault occurred at a single location [53]. However, it is still very challenging for this method [53] to resist glitch injection attack on computation units as multiple faults can occur simultaneously at diverse locations within the highly parallel PE array.

The vulnerability exploited by our attack and imperceptible misclassification [23] is the separate clock/voltage domain of

PE array used to accelerate DNN computations. There are several straightforward solutions to overcome this exploitation but they all come with a non-negligible loss of performance, power efficiency and productivity. If the DNN runs on a single clock/power domain, disrupting the clock or power supply can cause a denial of service instead of a misclassification, which can be easily detected and might trigger the intruder control system. However, eliminating multiple clock and power domains indirectly gets rid of DVFS, which is one of the most effective energy-quality scaling techniques used for hardware acceleration. Asynchronous design or globally asynchronous but locally synchronous design can possibly increase the difficulty of inducing timing violation accurately into a specific layer. However, the problem with self-time designs is productivity. They are not popular, not adaptable to reconfigurable platform, not easily testable, and not as well supported by existing design flow. The accelerator IPs, if available, may not be system-integration friendly. Obfuscation of model structure can also increase the cost of side-channel extraction of structural information to assist layer-aware glitch injection. This countermeasure adds non-trivial hardware cost and de-obfuscation performance overhead. Fault-tolerance techniques, such as triple modular redundancy [54] and randomization based reconfigurable PE [55], may be used to increase hardware robustness against deliberate or transient faults without sacrificing parallel processing and DVFS. However, they burn more hardware and power.

VI. CONCLUSION

In this paper, we introduce a versatile glitch attack on deep learning accelerator to misclassify a target in variational scenes. Our attack induces transient faults directly into the DNN computational process by injecting glitches into the clock signal of the PE array. Only one sample of the target in a fixed scene is sufficient to derive the attack patterns with the desirable glitch intensity on the specific layer. This is in stark contrast to the physical adversarial examples, which require a set of samples to profile an attack. A secretive mode and a robust mode of attack are demonstrated on FPGA-based deep learning accelerator using the LGA and SLA patterns, respectively. Various environmental conditions including 25 viewing angles (-60° to 60°), 24 light directions and 12 configurations of illumination colors on 118 objects have been evaluated. Our LGA has achieved 55.1% $\sim$ 71.7% attack success rate on four representative ImageNet models by injecting two glitches in ten thousand to a million clock cycles in one inference. Its success rate surpasses inconspicuous adversarial examples by 16.3% $\sim$ 63.5%. With SLA, the attack success rate has increased to above 96.2% on all four models. From the different error propagation behaviors through all layers of the DNN between adversarial examples and fault injection attack, process-oriented falsification is shown to be more effective than tampering the input or the physical target.

REFERENCES

- [1] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proc. IEEE*, vol. 107, no. 8, pp. 1738–1762, Aug. 2019.

- [2] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proc. IEEE*, vol. 105, no. 12, pp. 2295–2329, Dec. 2017.
- [3] P. Gysel, J. Pimentel, M. Motamedi, and S. Ghiasi, "Ristretto: A framework for empirical study of resource-efficient inference in convolutional neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 11, pp. 5784–5789, Nov. 2018.
- [4] C. Szegedy *et al.*, "Intriguing properties of neural networks," presented at the Int. Conf. Learning Representations (ICLR), Banff, AB, Canada, Apr. 2014.
- [5] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," presented at the Int. Conf. Learning Represent. (ICLR), San Diego, CA, USA, May 2015.
- [6] A. Kurakin, I. J. Goodfellow, and S. Bengio, "Adversarial machine learning at scale," presented at the Int. Conf. Learn. Represent. (ICLR), Toulon, France, Apr. 2017.
- [7] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *Proc. IEEE Symp. Secur. Privacy (SP)*, San Jose, CA, USA, May 2017, pp. 39–57.
- [8] P.-Y. Chen, Y. Sharma, H. Zhang, J. Yi, and C.-J. Hsieh, "EAD: Elastic-net attacks to deep neural networks via adversarial examples," in *Proc. 32nd AAAI Conf. Artif. Intell.*, New Orleans, LA, USA, Feb. 2018, pp. 10–17.
- [9] J. Lu, H. Sibai, E. Fabry, and D. Forsyth, "NO need to worry about adversarial examples in object detection in autonomous vehicles," 2017, *arXiv:1707.03501*. [Online]. Available: <http://arxiv.org/abs/1707.03501>
- [10] M. Sharif, S. Bhagavatula, L. Bauer, and M. K. Reiter, "Accessorize to a crime: Real and stealthy attacks on state-of-the-art face recognition," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Vienna, Austria, Oct. 2016, pp. 1528–1540.
- [11] K. Eykholt *et al.*, "Robust physical-world attacks on deep learning visual classification," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Salt Lake City, UT, USA, Jun. 2018, pp. 1625–1634.
- [12] J. Li, F. Schmidt, and Z. Kolter, "Adversarial camera stickers: A physical camera-based attack on deep learning systems," in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, Long Beach, CA, USA, vol. 97, Jun. 2019, pp. 3896–3904.
- [13] Y. Zhao, H. Zhu, R. Liang, Q. Shen, S. Zhang, and K. Chen, "Seeing isn't believing: Towards more robust adversarial attack against real world object detectors," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, London, U.K., Nov. 2019, pp. 1989–2004.
- [14] A. Athalye, L. Engstrom, A. Ilyas, and K. Kwok, "Synthesizing robust adversarial examples," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, Stockholm, Sweden, Jul. 2018, pp. 284–293.
- [15] K. Eykholt *et al.*, "Physical adversarial examples for object detectors," in *Proc. 12th USENIX Workshop Offensive Technol. (WOOT)*, Baltimore, MD, USA, Aug. 2018, pp. 1–10.
- [16] J. Breier, X. Hou, D. Jap, L. Ma, S. Bhasin, and Y. Liu, "Practical fault attack on deep neural networks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA, Oct. 2018, pp. 2204–2206.
- [17] Y. Liu, L. Wei, B. Luo, and Q. Xu, "Fault injection attack on deep neural network," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Irvine, CA, USA, Nov. 2017, pp. 131–138.
- [18] P. Zhao, S. Wang, C. Gongye, Y. Wang, Y. Fei, and X. Lin, "Fault sneaking attack: A stealthy framework for misleading deep neural networks," in *Proc. 56th Annu. Design Autom. Conf.*, Las Vegas, NV, USA, Jun. 2019, pp. 165:1–165:6.
- [19] J. Clements and Y. Lao, "Hardware trojan design on neural networks," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Sapporo, Japan, May 2019, pp. 1–5.
- [20] Y. Zhao *et al.*, "Memory trojan attack on neural network accelerators," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Grenoble, France, Mar. 2019, pp. 1415–1420.
- [21] M. M. Alam, S. Tajik, F. Ganji, M. Tehranipoor, and D. Forte, "RAM-JAM: Remote temperature and voltage fault attack on FPGAs using memory collisions," in *Proc. Workshop Fault Diagnosis Tolerance Cryptogr. (FDTC)*, Atlanta, GA, USA, Aug. 2019, pp. 48–55.
- [22] S. Hong, P. Frigo, Y. Kaya, C. Giuffrida, and T. Dumitras, "Terminal brain damage: Exposing the graceless degradation in deep neural networks under hardware fault attacks," in *Proc. 28th USENIX Secur. Symp.*, Santa Clara, CA, USA, Aug. 2019, pp. 497–514.
- [23] W. Liu, C.-H. Chang, F. Zhang, and X. Lou, "Imperceptible misclassification attack on deep learning accelerator by glitch injection," in *Proc. 57th ACM/IEEE Design Autom. Conf. (DAC)*, Jul. 2020, pp. 1–6.
- [24] D. Su, H. Zhang, H. Chen, J. Yi, P.-Y. Chen, and Y. Gao, "Is robustness the cost of accuracy?—a comprehensive study on the robustness of 18 deep image classification models," in *Proc. Euro. Conf. Comput. Vis. (ECCV)*, Munich, Germany, Sep. 2018, pp. 631–648.
- [25] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 770–778.
- [26] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. A. Alemi, "Inception-v4, inception-resnet and the impact of residual connections on learning," in *Proc. AAAI Conf. Artif. Intell.*, 2017, pp. 4278–4284.
- [27] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Salt Lake City, UT, USA, Jun. 2018, pp. 4510–4520.
- [28] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," presented at the Int. Conf. Learning Represent. (ICLR), San Diego, CA, USA, May 2014.
- [29] J.-M. Geusebroek, G. J. Burghouts, and A. W. M. Smeulders, "The Amsterdam library of object images," *Int. J. Comput. Vis.*, vol. 61, no. 1, pp. 103–112, Jan. 2005.
- [30] Y. LeCun, Y. Bengio, and G. E. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015.
- [31] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.
- [32] N. P. Jouppi *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proc. ACM/IEEE 44th Annu. Int. Symp. Comput. Archit. (ISCA)*, Toronto, ON, Canada, Jun. 2017, pp. 1–12.
- [33] B. Moons and M. Verhelst, "An energy-efficient precision-scalable ConvNet processor in 40-nm CMOS," *IEEE J. Solid-State Circuits*, vol. 52, no. 4, pp. 903–914, Apr. 2017.
- [34] *DPU for Convolutional Neural Network v3.0*, Xilinx, San Jose, CA, USA, 2019.
- [35] Y. Ma, Y. Cao, S. Vruthula, and J.-S. Seo, "Optimizing the convolution operation to accelerate deep neural networks on FPGA," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 26, no. 7, pp. 1354–1367, Jul. 2018.
- [36] S.-M. Moosavi-Dezfooli, A. Fawzi, O. Fawzi, and P. Frossard, "Universal adversarial perturbations," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Honolulu, HI, USA, Jul. 2017, pp. 86–94.
- [37] M. Agoyan, J.-M. Dutertre, D. Naccache, B. Robisson, and A. Tria, "When clocks fail: On critical paths and clock faults," in *Proc. Smart Card Res. Adv. Appl.*, Passau, Germany, Apr. 2010, pp. 182–193.
- [38] A. Tang, S. Sethumadhavan, and S. Stolfo, "CLKSCREW: Exposing the perils of security-oblivious energy management," in *Proc. 26th USENIX Secur. Symp.*, Vancouver, BC, Canada, Aug. 2017, pp. 1057–1074.
- [39] P. Qiu, D. Wang, Y. Lyu, and G. Qu, "VoltJockey: Breaching TrustZone by software-controlled voltage manipulation over multi-core frequencies," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, London, U.K., Nov. 2019, pp. 195–209.
- [40] K. Murdock, D. Oswald, F. D. Garcia, J. Van Bulck, D. Gruss, and F. Piessens, "Plundervolt: Software-based fault injection attacks against Intel SGX," in *Proc. IEEE Symp. Secur. Privacy (SP)*, San Francisco, CA, USA, May 2020, pp. 1466–1482.
- [41] Z. Kenjar, T. Frassetto, D. Gens, M. Franz, and A.-R. Sadeghi, "V0ltpwn: Attacking X86 processor integrity from software," in *Proc. 29th USENIX Secur. Symp.*, Aug. 2020, pp. 1445–1461.
- [42] P. Qiu, D. Wang, Y. Lyu, and G. Qu, "VoltJockey: Breaking SGX by software-controlled voltage-induced hardware faults," in *Proc. Asian Hardw. Oriented Secur. Trust Symp. (AsianHOST)*, Xian, China, Dec. 2019, pp. 1–6.
- [43] M. Sabbagh, Y. Fei, and D. Kaeli, "A novel GPU overdrive fault attack," in *Proc. 57th ACM/IEEE Design Autom. Conf. (DAC)*, Jul. 2020, pp. 1–6.
- [44] W. Hua, Z. Zhang, and G. E. Suh, "Reverse engineering convolutional neural networks through side-channel information leaks," in *Proc. 55th ACM/ESDA/IEEE Design Autom. Conf. (DAC)*, San Francisco, CA, USA, Jun. 2018, pp. 4:1–4:6.
- [45] N. F. Ghalaty, B. Yuze, M. Taha, and P. Schaumont, "Differential fault intensity analysis," in *Proc. Workshop Fault Diagnosis Tolerance Cryptogr.*, Busan, South Korea, Sep. 2014, pp. 49–58.
- [46] K. Ramezanpour, P. Ampadu, and W. Diehl, "Fault intensity map analysis with neural network key distinguisher," in *Proc. 3rd ACM Workshop Attacks Solutions Hardw. Secur. Workshop (ASHES)*, London, U.K., 2019, pp. 33–42.

- [47] M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Zurich, Switzerland, Sep. 6-12, 2014, pp. 818-833.
- [48] *Deep Neural Network Development Kit V3.1*, Xilinx, San Jose, CA, USA, 2019.
- [49] A. Arnab, O. Miksik, and P. H. S. Torr, "On the robustness of semantic segmentation models to adversarial attacks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Salt Lake City, UT, USA, Jun. 2018, pp. 888-897.
- [50] M. Esmailpour, P. Cardinal, and A. L. Koerich, "A robust approach for securing audio classification against adversarial attacks," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 2147-2159, 2020.
- [51] B. Liang, H. Li, M. Su, X. Li, W. Shi, and X. Wang, "Detecting adversarial image examples in deep neural networks with adaptive noise reduction," *IEEE Trans. Dependable Secure Comput.*, early access, Oct. 2019, doi: [10.1109/TDSC.2018.2874243](https://doi.org/10.1109/TDSC.2018.2874243).
- [52] J. Lin, C. Gan, and S. Han, "Defensive quantization: When efficiency meets robustness," presented at the Int. Conf. Learning Represent. (ICLR), New Orleans, LA, USA, May 2019.
- [53] M. Alam *et al.*, "Enhancing fault tolerance of neural networks for security-critical applications," 2019, *arXiv:1902.04560*. [Online]. Available: <http://arxiv.org/abs/1902.04560>
- [54] J. M. Johnson and M. J. Wirthlin, "Voter insertion algorithms for FPGA designs using triple modular redundancy," in *Proc. 18th Annu. ACM/SIGDA Int. Symp. Field Program. Gate Arrays (FPGA)*, Monterey, CA, USA, 2010, pp. 249-258.
- [55] B. Wang, L. Liu, C. Deng, M. Zhu, S. Yin, and S. Wei, "Against double fault attacks: Injection effort model, space and time randomization based countermeasures for reconfigurable array architecture," *IEEE Trans. Inf. Forensics Security*, vol. 11, no. 6, pp. 1151-1164, Jun. 2016.



Chip-Hong Chang (Fellow, IEEE) received the B.Eng. (Hons.) degree from the National University of Singapore in 1989, and the M.Eng. and Ph.D. degrees from Nanyang Technological University (NTU), Singapore, in 1993 and 1998, respectively.

He is currently an Associate Professor with the School of Electrical and Electronic Engineering (EEE), NTU. He held joint appointments with the university as an Assistant Chair of Alumni from 2008 to 2014, the Deputy Director of the Center for High Performance Embedded Systems from 2000 to 2011, and the Program Director of the Center for Integrated Circuits and Systems from 2003 to 2009. He has coedited four books, published 13 book chapters, more than 100 international journal articles (more than 70 are in IEEE) and more than 180 refereed international conference papers (mostly in IEEE), and delivered over 40 colloquia. His current research interests include hardware security, unconventional number systems, and low-power and fault-tolerant digital signal processing algorithms and architectures. He is a fellow of IET and a Distinguished Lecturer of IEEE Circuits and Systems Society from 2018 to 2019. He currently serves as a Senior Area Editor of IEEE TRANSACTIONS ON INFORMATION FORENSIC AND SECURITY and an Associate Editor of the IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS I: REGULAR PAPERS and IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS. He also served as an Associate Editor of the IEEE TRANSACTIONS ON INFORMATION FORENSIC AND SECURITY and IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGN OF INTEGRATED CIRCUITS AND SYSTEMS from 2016 to 2019, IEEE ACCESS from 2013 to 2019, IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS I: REGULAR PAPERS from 2010 to 2013, *Integration, the VLSI Journal* from 2013 to 2015, *Journal of Hardware and Systems Security* (Springer) from 2016 to 2020, and *Microelectronics Journal* from 2014 to 2020. He guest edited several special issues and served in the organizing and technical program committee for more than 60 international conferences (mostly IEEE).



Wenye Liu (Graduate Student Member, IEEE) received the B.S. degree in microelectronics from Shenzhen University, China, in 2014, the B.S. degree in physics from Umea University, Sweden, in 2014, and the M.S. degree in IC design engineering from The Hong Kong University of Science and Technology. He is currently pursuing the Ph.D. degree with the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore. His current research interests include hardware security, machine learning accelerator, and fault injection attack.



Fan Zhang (Member, IEEE) received the Ph.D. degree from the Department of Computer Science and Engineering, University of Connecticut, in 2012. He is currently a Full Professor with the College of Computer Science and Technology, Zhejiang University. His research interests include system security, hardware security, cryptography, and computer architecture.